

Pipeline CI/CD Seguro para la Detección Automática de Vulnerabilidades Mediante Inteligencia Artificial y Minería de Datos

Asmal Kevin
Fernando Tipán

Universidad de las Fuerzas Armadas ESPE
Carrera de Ingeniería en Software

Resumen—El este proyecto tuvo como objetivo diseñar e implementar un pipeline CI/CD seguro capaz de poder detectar automáticamente vulnerabilidades en el código fuente mediante técnicas de inteligencia artificial basadas en la minería de datos, para ello se desarrolló un backend utilizando Flask, se entrenó un modelo de clasificación utilizando Logistic Regression y se empleó la técnica TF-IDF para la extracción de las características del código fuente.

La solución implementada integra los principios de DevSecOps y Shift-Left Security, permitiendo incorporar los controles de seguridad desde etapas tempranas del desarrollo, se automatizaron los procesos de validación mediante GitHub Actions, pruebas con Pytest, notificaciones por Telegram y el despliegue continuo en Render. Los resultados que fueron obtenidos demuestran que la integración de inteligencia artificial dentro de procesos CI/CD nos permite mejorar la calidad del software, reducir vulnerabilidades y fortalecer la seguridad durante todo el ciclo de vida de desarrollo.

Index Terms—DevSecOps, SSDLC, CI/CD, Inteligencia Artificial, Seguridad de Software, Logistic Regression, TF-IDF.

I. Introducción

La transformación digital ha impulsado la creación de aplicaciones cada vez más complejas y conectadas a Internet, lo que ha incrementando la superficie de ataque y la posibilidad de explotación de vulnerabilidades, los controles de seguridad se realizaban al final del proceso de desarrollo lo que genera costos elevados cuando se detectaban errores críticos. Para solucionar esta problemática surgieron los enfoques como DevSecOps y Shift-Left Security, cuyo propósito es poder integrar la seguridad desde las primeras fases del ciclo de vida del software, con estas metodologías promueven la automatización de controles de seguridad, pruebas y validaciones continuas.

En este proyecto se desarrolló una solución que combina integración continua, despliegue continuo e inteligencia artificial para poder clasificar automáticamente fragmentos de código fuente como seguros o vulnerables. El sistema permite bloquear los cambios inseguros antes de que alcancen entornos de producción, contribuyendo a mejorar la calidad y seguridad del software desarrollado.

I-A. Importancia de la Seguridad en el Desarrollo Moderno

En la actualidad las organizaciones dependen cada vez más de las aplicaciones web, servicios en la nube y sistemas distribuidos para poder gestionar información crítica. Esta dependencia tecnológica ha incrementado la exposición a amenazas cibernéticas, haciendo que la seguridad del software se convierta en un requisito fundamental durante todo el proceso de desarrollo, los ataques informáticos actuales aprovechan vulnerabilidades presentes en el código fuente, en los errores de configuración y la deficiencias en los procesos de despliegue, como consecuencia las empresas han comenzado a adoptar enfoques preventivos que les permitan detectar los problemas de seguridad antes de que el software llegue a producción.

La incorporación de inteligencia artificial dentro de los procesos DevSecOps representa una evolución significativa en la automatización de controles de la seguridad, permitiendonos identificar patrones de riesgo de manera más eficiente y reduciendo el tiempo requerido para realizar revisiones manuales.

II. Objetivos

II-A. Objetivo General

Diseñar e implementar un pipeline CI/CD seguro que nos ayude a integrar un modelo de inteligencia artificial basado en minería de datos para detectar automáticamente las vulnerabilidades en código fuente antes de su despliegue a producción.

II-B. Objetivos Específicos

- Desarrollar un backend seguro utilizando Flask.
- Implementar un modelo de Machine Learning basado en Logistic Regression.
- Utilizar TF-IDF para la extracción de características del código fuente.
- Automatizar procesos de integración y despliegue continuo mediante GitHub Actions.

- Implementar mecanismos de notificación mediante Telegram.
- Aplicar principios SSDLC y DevSecOps durante el desarrollo.
- Garantizar que únicamente código validado alcance producción.

III. Marco Teórico

III-A. Desarrollo de Software Seguro

El desarrollo de software seguro consiste en aplicar prácticas, metodologías y controles orientados a poder minimizar vulnerabilidades durante todo el ciclo de vida de una aplicación, el principal objetivo es preservar la confidencialidad, integridad y la disponibilidad de la información.

III-B. SSDLC

El Secure Software Development Life Cycle (SSDL) es una evolución del ciclo de vida tradicional del software que incorpora actividades de la seguridad en cada fase del desarrollo.

Las etapas principales incluyen:

- Planificación.
- Análisis de riesgos.
- Diseño seguro.
- Implementación segura.
- Pruebas de seguridad.
- Despliegue controlado.
- Mantenimiento y monitoreo.

III-C. DevSecOps

DevSecOps integra las prácticas de seguridad dentro de los procesos DevOps, promoviendo la automatización de los controles de seguridad en los pipelines de integración y despliegue continuo.

Entre sus beneficios destacan:

- Detección temprana de vulnerabilidades.
- Automatización de revisiones.
- Reducción de riesgos operativos.
- Mayor calidad del software.

III-D. Shift-Left Security

Shift-Left Security nos propone trasladar las actividades de seguridad hacia las primeras fases del desarrollo para reducir el costo de la corrección de vulnerabilidades.

III-E. CI/CD

La Integración Continua (CI) consiste en validar automáticamente los cambios realizados por los desarrolladores, por su parte el Despliegue Continuo (CD) automatiza la liberación de nuevas versiones hacia los ambientes productivos.

III-F. TF-IDF

TF-IDF (Term Frequency-Inverse Document Frequency) es una técnica ampliamente utilizada en los procesamiento de lenguaje natural para representar el texto mediante vectores numéricos.

En este proyecto se utilizó para poder transformar fragmentos del código fuente en características numéricas procesables por algoritmos de Machine Learning.

III-G. Logistic Regression

Logistic Regression es un algoritmo supervisado de clasificación binaria ampliamente utilizado debido a su eficiencia, además de su simplicidad e interpretabilidad.

Su función principal consiste en estimar la probabilidad de pertenencia de una observación a una determinada clase.

III-H. GitHub Actions

GitHub Actions es una plataforma de automatización que nos permite ejecutar workflows asociados a eventos dentro de un repositorio.

Fue utilizada para poder implementar el pipeline CI/CD seguro del proyecto.

III-I. Telegram Bot

Telegram proporciona una API que nos permite desarrollar bots para automatizar notificaciones y procesos de comunicación.

En este proyecto se utilizó para poder informar eventos críticos del pipeline.

IV. Metodología

El desarrollo se realizó aplicando los principios SSDLC y DevSecOps.

Las fases implementadas fueron las siguientes:

1. Análisis de requisitos.
2. Diseño de arquitectura.
3. Desarrollo del backend.
4. Entrenamiento del modelo de IA.
5. Integración del pipeline.
6. Validación mediante pruebas.
7. Despliegue en producción.
8. Monitoreo y notificaciones.

V. Desarrollo del Proyecto

V-A. Arquitectura del Sistema

La solución fue diseñada utilizando una arquitectura por capas para facilitar el mantenimiento el desacoplamiento de los componentes y la escalabilidad del sistema ante un incremento

en el flujo de peticiones. Esta separación estructural nos garantiza que las modificaciones en la lógica de detección o en el almacenamiento no impacten directamente en la disponibilidad del servicio.

V-A1. Capa de Presentación: Implementada mediante el microframework Flask, es la responsable de poder exponer los endpoints de la API REST y gestionar de manera eficiente las solicitudes HTTP entrantes, es la que se encarga de recibir los fragmentos de código fuente enviados por los clientes (como los webhooks de GitHub Actions), poder validar el formato de los datos de entrada y poder retornar las respuestas en formato JSON con los códigos de estado correspondientes.

V-A2. Capa de Negocio: Constituye el núcleo analítico del sistema y contiene la lógica encargada de la clasificación del código fuente y la validación de vulnerabilidades, en esta capa se orquestan los procesos de preprocesamiento de texto, la vectorización de las cadenas de caracteres y la ejecución del modelo predictivo para dictaminar si un bloque de código representa una de las amenazas potenciales o también si cumple con los estándares de seguridad requeridos.

V-A3. Capa de Datos: Representa el soporte persistente de la aplicación es el que se encarga de almacenar de forma segura los datasets de entrenamiento, los modelos de Machine Learning previamente serializados (archivos de extensión .pkl o similares) y los recursos estadísticos utilizados por el vectorizador TF-IDF para la extracción de las características del código fuente.

V-B. Backend Seguro

El backend fue desarrollado utilizando Flask y siguiendo rigurosas buenas prácticas de programación segura orientadas a poder mitigar los riesgos identificados en las metodologías de la industria como el OWASP Top 10, se priorizó el desarrollo defensivo para poder evitar que el propio sistema de detección se convirtiese en un vector de ataque dentro de la infraestructura corporativa.

Se implementaron unos mecanismos avanzados para la validación estricta de las entradas mediante expresiones regulares, la sanitización de cadenas para poder prevenir ataques de inyección y una gestión segura de las configuraciones que evita la exposición de credenciales en el código fuente. Asimismo se estructuró una separación de responsabilidades clara y un manejo controlado de los errores y excepciones que impide la fuga de información sensible a través de las trazas o mensajes de depuración del servidor en entornos de producción. Se implementaron mecanismos para:

- Validación de entradas.
- Gestión segura de configuraciones.
- Separación de responsabilidades.
- Manejo controlado de errores.

V-C. Modelo de Inteligencia Artificial

El modelo fue entrenado utilizando el Scikit-Learn.
Proceso seguido:

1. Carga del dataset.
2. Limpieza de datos.
3. Transformación mediante TF-IDF.
4. División entrenamiento/prueba.
5. Entrenamiento Logistic Regression.
6. Evaluación del rendimiento.
7. Exportación del modelo.

V-D. Pipeline CI/CD

El pipeline fue implementado utilizando el GitHub Actions.

V-D1. Etapa 1: Revisión de Seguridad: Se ejecuta automáticamente al crear un Pull Request desde la rama dev hacia test.

Actividades:

- Análisis del código.
- Clasificación mediante IA.
- Generación de alertas.
- Creación automática de Issues.
- Etiquetado automático.

V-D2. Etapa 2: Pruebas: Se ejecutan pruebas unitarias utilizando Pytest.

V-D3. Etapa 3: Despliegue: Cuando todas las validaciones son satisfactorias la aplicación se despliega automáticamente en Render.

VI. Mecanismos de Seguridad Implementados

- Branch Protection Rules.
- Pull Requests obligatorios.
- Revisión automática mediante IA.
- Validación mediante pruebas automatizadas.
- Gestión de secretos mediante GitHub Secrets.
- Etiquetas automáticas.
- Issues automáticas.
- Notificaciones Telegram.
- Despliegue controlado.

VII. Resultados

Los resultados que fueron obtenidos evidencian el correcto funcionamiento del pipeline seguro.

VII-A. Resultados del Modelo

```
(venv) PS D:\Desarrollo de S.Seguro\Segundo Parcial\ProyectoSegundoParcial\Proyecto2Parcial_DesarrolloSeguroSoftware> python src
/ML/train.py
Cargando dataset...
Extrayendo features manuales...
Generando TF-IDF...
Dividiendo dataset...
Entrenando LogisticRegression...
D:\Desarrollo de S.Seguro\Segundo Parcial\ProyectoSegundoParcial\Proyecto2Parcial_DesarrolloSeguroSoftware\venv\Lib\site-package
sklearn\linear_model\_logistic.py:486: ConvergenceWarning: lbfgs failed to converge after 3000 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=3000).
You might also want to scale the data as shown in:
https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
n_iter_i = _check_optimize_result(
Accuracy: 0.9676
=====
              precision    recall  f1-score   support

     0       0.99      0.94      0.97      865
     1       0.94      0.99      0.97      865

 accuracy      0.97      0.97      0.97      1730
 macro avg      0.97      0.97      0.97      1730
weighted avg      0.97      0.97      0.97      1730

Modelo guardado.
(venv) PS D:\Desarrollo de S.Seguro\Segundo Parcial\ProyectoSegundoParcial\Proyecto2Parcial_DesarrolloSeguroSoftware>
```

Figura 1. Resultados del modelo de IA

VII-B. Validación del Pipeline

Se verificó exitosamente:

- Detección automática de vulnerabilidades.
- Bloqueo de Pull Requests inseguros.
- Creación automática de Issues.
- Envío de notificaciones.
- Ejecución de pruebas.
- Despliegue automático.

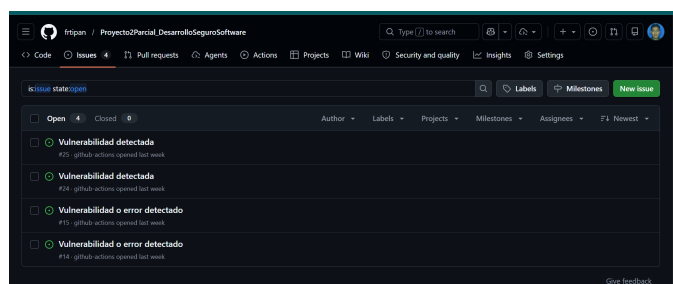


Figura 2. Resultados del pipeline

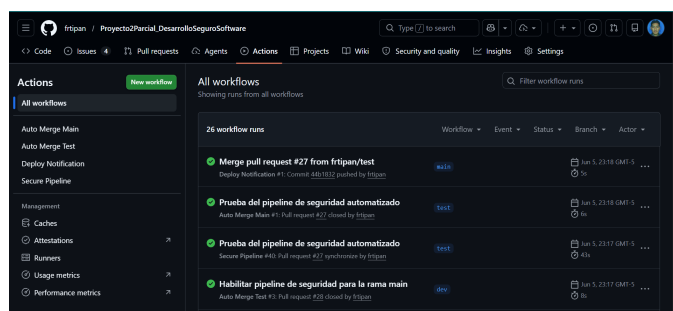


Figura 3. Resultados de Issues

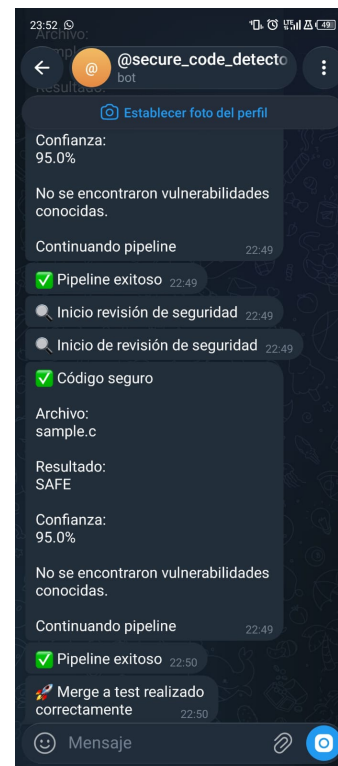


Figura 4. Resultados de envío de notificaciones

VIII. Discusión

La integración de las técnicas de Machine Learning dentro de los procesos DevSecOps representa una alternativa disruptiva frente a los métodos tradicionales de revisión estática ya que la combinación de TF-IDF y Logistic Regression nos demostró una notable capacidad para poder generalizar los patrones asociados a brechas de seguridad latentes, optimizando los tiempos de revisión y consolidando el enfoque de *Shift-Left Security*. No obstante existe un compromiso estructural (*trade-off*) entre la velocidad de inferencia y la complejidad abstracta del modelo, si bien Logistic Regression ofrece un rendimiento computacional sumamente ligero e idóneo para los entornos de integración continua (CI/CD) donde se prioriza la agilidad del flujo, su naturaleza lineal podría presentar las limitaciones frente a las dependencias semánticas complejas que requerirán futuras exploraciones con las arquitecturas de Deep Learning o grafos de flujo de control, la automatización integral del pipeline nos ayudo a reducir drásticamente el sesgo operativo y la intervención humana lo que ayudo a mitigar omisiones por fatiga mediante un ecosistema de aislamiento y retroalimentación inmediata que propicia la observabilidad y una cultura de desarrollo seguro en los equipos de ingeniería.

IX. Conclusiones

El desarrollo de este proyecto nos permitió validar con éxito la implementación de un pipeline CI/CD robusto bajo

el enfoque DevSecOps demostrando que la sinergia entre el vectorizador TF-IDF y el algoritmo Logistic Regression nos constituye una solución eficaz y de bajo costo computacional para interceptar código vulnerable antes de que alcance entornos productivos, el uso de GitHub Actions como orquestador principal y la integración de un bot de uso de Telegram evidenciaron que la automatización de los flujos de trabajo de seguridad erradica de forma drástica los errores derivados de las verificaciones manuales y optimiza la observabilidad del sistema en tiempo real, la adopción transversal de la metodología SSDLC consolidó la premisa de que la seguridad de software debe ser un eje preventivo y estructural logrando mitigar los riesgos operativos identificados en los estándares como el OWASP Top 10 y poder garantizar que únicamente el código rigurosamente validado sea desplegado en plataformas como Render.

X. Recomendaciones

Para futuras líneas de investigación y optimización del sistema se sugiere expandir sustancialmente el volumen y la diversidad del dataset de entrenamiento utilizando repositorios de referencia actualizados para poder mitigar el margen de falsos negativos, al mismo tiempo que se evoluciona la fase de extracción de las características sustituyendo el análisis estadístico de TF-IDF por las técnicas contextuales como los Grafos de Sintaxis Abstracta (AST) o los *Code Embeddings*, resulta fundamental poder ampliar la cobertura de pruebas automatizadas mediante la combinación de análisis estático (SAST) complementario con las herramientas de análisis dinámico (DAST), lo que nos facilita la estructuración de una estrategia de defensa en capas que reduzca la fricción en la capa de negocio de Flask, se recomienda implementar algunos mecanismos avanzados de telemetría en producción dentro de la infraestructura de Render para poder recolectar datos anómalos reales, habilitando un flujo de reentrenamiento cíclico que refine la precisión predictiva del modelo de inteligencia artificial de forma continua.

Referencias

- [1] OWASP Foundation, *OWASP Top 10*, 2025.
- [2] GitHub, *GitHub Actions Documentation*, 2025.
- [3] Pallets Team, *Flask Documentation*, 2025.
- [4] Scikit-Learn Developers, *Scikit-Learn User Guide*, 2025.
- [5] J. Willis, *DevSecOps Fundamentals*, ACM Press, 2024.